

Constrained Discrete Diffusion Models for Graph Generation

Nico Ohler

Seminar Project Machine Learning (708.416)

Institute of Machine Learning and Neural Computation, TU Graz

Winter Semester 2025

1 Introduction

The practical applications of diffusion models are abundant, having achieved impressive feats across various domains such as image generation (Ho et al., 2020), 3D modeling (Xue et al., 2024) and drug discovery (Alakhdar et al., 2024), with prior work having successfully adopted them to the discrete graph domain. Despite great success, there are still challenges that need to be overcome. Traditional graph diffusion approaches typically predict transition probabilities for node and edge attributes and thus suffer from limited scalability due to the quadratic complexity with respect to the number of nodes. Another important aspect of graph generation is constraint satisfaction, i.e., producing outputs that fulfill certain criteria.

The primary goal of this work is to address both these issues by investigating how well *SparseDiff* (Qin et al., 2025), a novel framework for efficient sampling of sparse graphs, handles different types of constraints and whether output quality can be improved by correcting intermediate outputs using *Projected Diffusion Models* (PDMs) (Christopher et al., 2024) as well as explicitly penalizing constraint violations via unlikelihood training (Welleck et al., 2019). For this, *SparseDiff* was extended with optional *projected diffusion* and an *unlikelihood training* procedure for fine-tuning.¹ With the modified architecture, three models were trained on a synthetic toy dataset, which implicitly imposes several constraints ranging from specific edge lengths to more global constraints such as connectivity and planarity. Their performance was assessed via custom metrics that track constraint violations for generated graphs.

The baseline *SparseDiff* model produces samples similar to those from the training data. Those graphs rarely contain edges with illegal lengths but occasionally disconnected components and frequently edge crossings, indicating that global constraints are significantly more difficult. To counteract this, unlikelihood training was employed to discourage edge crossings but no clear benefit over regular training could be observed. Regarding projected diffusion, correcting the output of the final denoising step leads to strictly better graphs that fulfill all constraints as expected. Projecting the output of every intermediate step, on the other hand, did not lead to any obvious improvements regarding structural similarity to the training data. The additional computational overhead and risks of handling out-of-distribution inputs make it hard to justify the use of projected diffusion for intermediate outputs without clear advantages.

2 Related Work

2.1 Graph Diffusion Models

Denoising Probabilistic Diffusion Models (DDPMs) construct a Markov chain, called forward diffusion, where the ground truth data is gradually diffused by adding noise until the data is indistinguishable from a well-known and easy-to-sample data distribution. During the reverse diffusion, a parameterized transition kernel learns to undo this process, which can later be used to generate entirely new samples based on this simpler distribution. While initial work primarily focused on DDPMs to iteratively remove continuous Gaussian noise during the denoising process, this approach proved to be problematic when working with discrete data. To address this issue, *Discrete Denoising Probabilistic Diffusion Models* (D3PMs) instead employ a sequence of conditional transition matrices that describe the likelihood

¹The source code is available at <https://github.com/NicoOhler/ConstrainedSparseDiff>.

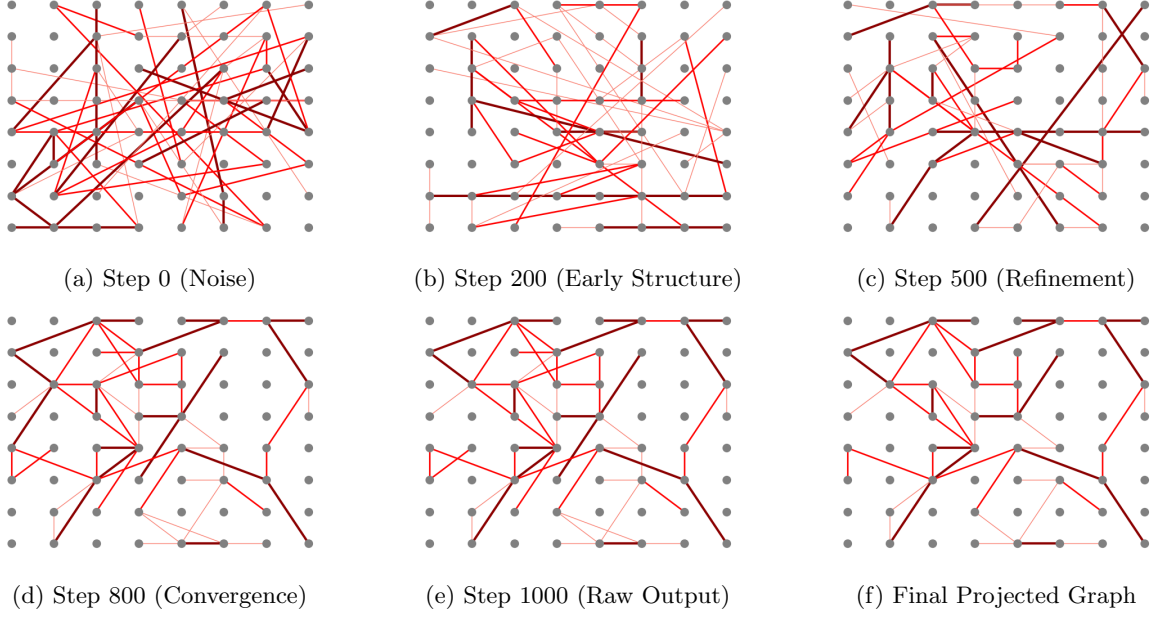


Figure 1: Graph generation over $T = 1000$ diffusion steps

of moving from one state to another (Austin et al., 2021). Recent research by Vignac et al. (2022) has successfully adapted D3PMs to generate new graphs, resembling the training data, by predicting transition probabilities for categorical edge and node attributes. Furthermore, they also leveraged classifier guidance for graph conditioning and trained a separate discriminator to predict whether the noisy graph is likely to fulfill certain desirable properties. This gradient can then be used to push the generation trajectory towards desirable regions. However, explicitly modeling dense edge transition matrices suffers from limited scalability, as the number of potential edges scales quadratically with the number of nodes, thus making it challenging to sample large graphs.

2.2 SparseDiff

In order to overcome these scaling issues, Qin et al. (2025) propose a new diffusion framework, *SparseDiff*, that utilizes the inherent sparsity typically present in large graphs.

Graphs are represented via a compact triplet $(\mathbf{E}, \mathbf{X}, \mathbf{Y})$, consisting of an edge list of node indices $\mathbf{E}$, one-hot encoded node features $\mathbf{X}$ and edge features $\mathbf{Y}$, which is significantly more memory-efficient for sparse graphs compared to dense adjacency matrices.

The forward diffusion, denoising and inference processes were all adapted to improve the space complexity and ensure that sampled graphs remain sparse throughout the whole process. Similar to existing discrete diffusion models, the forward diffusion operates on edge and node distributions instead of diffusing the graphs directly:

$$q(G^t|G^{t-1}) = (\mathbf{X}\mathbf{Q}_{\mathbf{X}}^t, \mathbf{Y}\mathbf{Q}_{\mathbf{Y}}^t) \quad (1)$$

where $\mathbf{Q}^t$ represents the Markov transition matrix for the diffusion step t .

The distributions $\mathbf{Q}_{\mathbf{X}}^t, \mathbf{Q}_{\mathbf{Y}}^t$ are obtained via interpolation:

$$\mathbf{Q}_{\mathbf{X}}^t = \alpha^t \mathbf{I} + (1 - \alpha^t) \mathbf{1}_a \mathbf{p}_{\mathbf{X}}^\top \quad (2)$$

$$\mathbf{Q}_{\mathbf{Y}}^t = \alpha^t \mathbf{I} + (1 - \alpha^t) \mathbf{1}_b \mathbf{p}_{\mathbf{Y}}^\top \quad (3)$$

Structure from G^{t-1} is incorporated by the identity matrix term, while the second term is responsible for adding noise in the form of the marginal distributions $\mathbf{p}_{\mathbf{X}}, \mathbf{p}_{\mathbf{Y}}$ of the training data, with $\alpha^t \in [0, 1]$ controlling the noise influence at step t .

Analogous to regular diffusion, the noisy distribution G^t can be directly obtained from the initial graph G :

$$q(G^t|G) = (\mathbf{X}\bar{\mathbf{Q}}_{\mathbf{X}}^t, \mathbf{Y}\bar{\mathbf{Q}}_{\mathbf{Y}}^t) \quad (4)$$

Algorithm 1 Sparse training at step t with the sparsity parameter λ

- 1: Given the clean graph $G = (\mathbf{E}^0, \mathbf{X}^0, \mathbf{Y}^0)$;
 - 2: Sample the noisy graph $G^t = (\mathbf{E}^t, \mathbf{X}^t, \mathbf{Y}^t)$;
 - 3: Sample query edges $\mathbf{E}_q$ of size $\lceil \lambda n^2 \rceil$;
 - 4: $\mathbf{E}_m \leftarrow \mathbf{E}^t \cup \mathbf{E}_q, \mathbf{Y}_m \leftarrow \mathbf{Y}^t \cup \mathbf{Y}_q$; ▷ Construct message-passing edges
 - 5: $G_m \leftarrow (\mathbf{E}_m, \mathbf{X}^t, \mathbf{Y}_m)$; ▷ Construct the message-passing graph
 - 6: $(\hat{\mathbf{P}}_q^{\mathbf{X}}, \hat{\mathbf{P}}_q^{\mathbf{Y}}) = \phi_\theta(G_m, \mathbf{E}_q)$; ▷ Predict the distribution of nodes and query edges
 - 7: optimizer.step($\text{CE}(\hat{\mathbf{X}}^0, \hat{\mathbf{P}}^{\mathbf{X}}) + \text{CE}(\mathbf{Y}_q^0, \hat{\mathbf{P}}_q^{\mathbf{Y}})$); ▷ Loss calculation
-

Reference: Qin et al. (2025)

Algorithm 2 Iterative inference at step t with the sparsity parameter λ

- 1: Initialize an empty graph G^{t-1} with unlabeled nodes $\mathbf{X}^{t-1}$ and no edges;
 - 2: Randomly divide all node pairs into $K = \lceil \frac{1}{\lambda} \rceil$ equal-sized chunks $\{C_0, \dots, C_{K-1}\}$;
 - 3: **for** k in range(K) **do**
 - 4: $\mathbf{E}_q \leftarrow C_k$; ▷ Set query edges
 - 5: $\mathbf{E}_m \leftarrow \mathbf{E}^t \cup \mathbf{E}_q$; ▷ Construct message-passing edges and its attributes $\mathbf{Y}_m$
 - 6: $G_m \leftarrow (\mathbf{E}_m, \mathbf{X}^t, \mathbf{Y}_m)$; ▷ Construct the message-passing graph
 - 7: $(\hat{\mathbf{P}}_q^{\mathbf{X}}, \hat{\mathbf{P}}_q^{\mathbf{Y}}) = \phi_\theta(G_m, \mathbf{E}_q)$; ▷ Predict the distribution of nodes and query edges
 - 8: $\hat{\mathbf{X}} = \text{Multinomial}(\hat{\mathbf{P}}^{\mathbf{X}}), \hat{\mathbf{Y}}_q = \text{Multinomial}(\hat{\mathbf{P}}_q^{\mathbf{Y}})$; ▷ Sample labels
 - 9: $\mathbf{X}^{t-1} \leftarrow \hat{\mathbf{X}}$; ▷ Assign node new labels
 - 10: $\mathbf{Y}^{t-1} \leftarrow \mathbf{Y}^{t-1} \cup \hat{\mathbf{Y}}_q[\hat{\mathbf{Y}}_q! = 0], \mathbf{E}^{t-1} \leftarrow \mathbf{E}^{t-1} \cup \hat{\mathbf{E}}_q[\hat{\mathbf{Y}}_q! = 0]$; ▷ Add ‘existing’ edges
 - 11: **end for**
-

Reference: Qin et al. (2025)

where the cumulative transition matrices are defined as:

$$\bar{\mathbf{Q}}_{\mathbf{X}}^t = \prod_{i=1}^t \mathbf{Q}_{\mathbf{X}}^i = \bar{\alpha}^t \mathbf{I} + (1 - \bar{\alpha}^t) \mathbf{1}_a \mathbf{p}_{\mathbf{X}}^\top \quad (5)$$

$$\bar{\mathbf{Q}}_{\mathbf{Y}}^t = \prod_{i=1}^t \mathbf{Q}_{\mathbf{Y}}^i = \bar{\alpha}^t \mathbf{I} + (1 - \bar{\alpha}^t) \mathbf{1}_b \mathbf{p}_{\mathbf{Y}}^\top \quad (6)$$

This choice of the marginal distribution differentiates *SparseDiff* from related work, favoring non-edges and thus encouraging graphs to remain sparse.

Furthermore, noisy graph distributions are computed in a memory-efficient manner by computing $\mathbf{Y} \bar{\mathbf{Q}}_{\mathbf{Y}}^t \in \mathbb{R}^{m \times b}$ only for existing edges and using the following approach for nonexistent edges. First, the number of new edges is drawn from a binomial distribution, then the node indices of each edge are uniformly sampled and finally their edge attributes are sampled from the multinomial distribution, based on the transition probabilities from non-edges to existing edge types. Doing so significantly reduces memory needs and lowers the space complexity to $\mathcal{O}(1 + m + 4mb + b^2 + 2n^2)$. For comparison, DiGress (Vignac et al., 2022) has a total space complexity of $\mathcal{O}(3n^2b + b^2)$.

Conventional graph diffusion models typically suffer from quadratic complexity during training as they operate on all possible edges. Relying on message-passing between existing edges enables the training of graphs with a larger amount of nodes, albeit at the cost of significantly degraded performance. Instead, the authors propose a sparse attention mechanism that restricts the use of convolutional attention layers to existing edges $\mathbf{E}^t$ of the noisy graph as well as a limited number of randomly selected query edges $\mathbf{E}_q$. In order to keep the computational complexity at an acceptable level, the number of query edges is controlled via a hyperparameter defined as $\lambda = \frac{|\mathbf{E}_q|}{n^2}$. Furthermore, the cross entropy loss is computed only on random query edges and thus effectively performs Stochastic Gradient Descent. Aside from that, several more optimization techniques are employed in order to keep the computational complexity in check while producing promising results.

During inference, an initial random graph is sampled from a categorical distribution based on the marginal probabilities of the node and edge attributes. Analogous to regular graph diffusion approaches, the final graph is generated by repeatedly denoising this initial graph. In order to control the memory complexity and avoid distribution shifts, all node pairs are split into disjoint chunks, having the same size as the query edges during training. These chunks are then fed one by one as input into the denoising

network, yielding a new distribution from which a cleaner version of the noisy graph is constructed. The relatively slow sampling speed, which arises due to having to apply the denoising model several times at each step, can be accelerated by skipping over intermediate diffusion steps.

2.3 Enforcing Constraints

Christopher et al. (2024) propose PDMs, which enforce hard constraints by correcting the proposed output from pretrained diffusion models via projection to the closest feasible point. This inference time correction guarantees valid output that fully respects all constraints. A major limitation of this approach is the reliance on the assumption that the corrected output remains fairly similar to the original. However, the projection could also generate intermediate output that is drastically different from that produced during training. Incorporating the projection into the training may solve this issue, although doing so would slow the training procedure, especially if the projection step cannot be carried out through efficient vectorized operations.

Anonymous (2025) provide an interesting conceptually similar approach in which edge transitions are masked if the resulting graph would violate the given constraints. For example, when enforcing connectivity, the removal of an edge, which would cause the graph to become disconnected, could be blocked by setting its transition probability to zero. While this requires validity checks during each diffusion step, the effective computational cost is often significantly lower since the graph of the previous step already satisfied all the constraints.

Unlikelihood training refers to explicitly penalizing undesired outputs and reducing their likelihood for future predictions. Welleck et al. (2019) introduced this concept and successfully applied it to discourage language models from producing repetitive text sequences. Their motivation was to overcome a major limitation of *Maximum Likelihood Estimation* (MLE), namely that oftentimes generated samples may have a high probability but nonetheless be inappropriate. In the context of text generation, a typical example might look like this: “Supported languages include French, Spanish, Italian, Spanish, ...”. While assigning relatively high probabilities to languages that have already occurred is reasonable, it is clearly wrong. For a similar reason, Zhu et al. (2022) utilized unlikelihood training to prevent motion forecasting models from predicting trajectories that are close to the ground truth but violate certain constraints, such as road boundaries. Whenever invalid outputs were identified, their log likelihood would be added to the loss as a penalty term. Unlikelihood training did not only drastically reduce the number of constraint violations but also improved the overall prediction accuracy.

While previous work successfully demonstrated the usefulness of classifier guidance and transition masking methods for constrained graph generation, this work will instead extend the *SparseDiff* architecture with projection as well as unlikelihood training and evaluate their effectiveness.

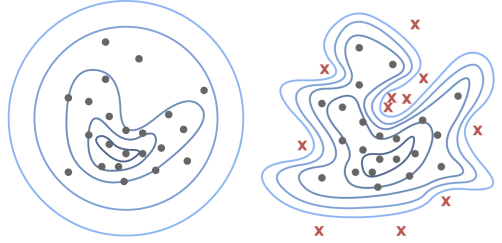


Figure 2: Illustrations from Regenwetter et al. (2024) highlighting how deliberate exposure to negative data can provide an informed fit to the true data distribution.

3 Experiments

3.1 Synthetic Dataset

A random walk algorithm was used to procedurally generate undirected, connected and planar graphs that live on a two-dimensional 8x8 coordinate grid. The synthetic dataset consists of 5000 such graphs, from which 1000 are reserved for testing and 800 for validation. Existing edges $((x_1, y_1), (x_2, y_2))$ are associated with a weight $w \in \{1, 2, 3\}$ and restricted to a length of

$$l = \sqrt{\Delta x^2 + \Delta y^2} \in \{1, \sqrt{2}, \sqrt{5}\} \quad \text{with} \quad \Delta x = |x_1 - x_2|, \Delta y = |y_1 - y_2|. \quad (7)$$

Intuitively, this can be thought of as the union of king and knight moves on a chessboard. These node coordinates and edge weights are then converted by *SparseDiff* (described in the subsequent section) into one-hot encoded node features $\mathbf{X} \in \mathbb{R}^{n \times 64}$ and edge features $\mathbf{Y} \in \mathbb{R}^{m \times 4}$, where the fourth edge feature represents nonexistent edges.

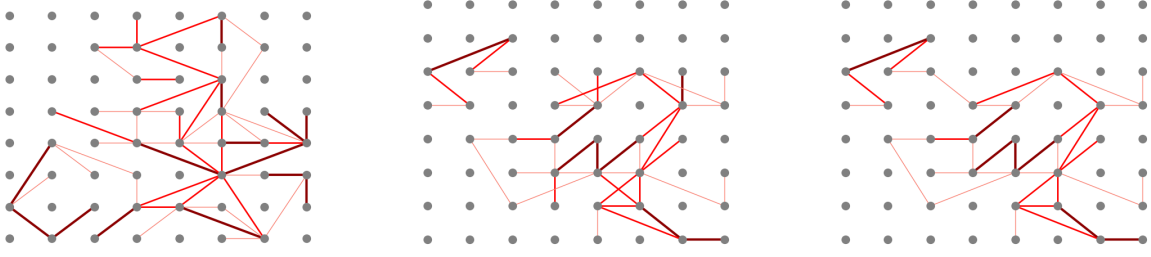


Figure 3: (a) a graph from the synthetic dataset, (b) a sampled graph which violates the connectivity and planarity constraints, (c) a corrected graph which satisfies all constraints after projection

3.2 SparseDiff Model Training and Modifications

For the experiment, the *SparseDiff* architecture, whose source code is publicly available, was adapted to the synthetic dataset and extended with an optional projection step analogous to PDMs, an optional unlikelihood training procedure and customized metrics to evaluate the quality of output graphs with respect to a set of predefined constraints: planarity, connectivity and certain edge lengths.

Additionally, *SparseDiff* was modified to keep the node attributes fixed since every graph contains each of the 64 node positions exactly once. While the denoising network still predicted new coordinates, they were simply ignored, and the training loss was thus limited to the cross entropy between edge transition probabilities.

Using this modified architecture, an initial baseline model Φ_{b100} was trained for 100 epochs on the synthetic training data, providing an initial impression of how well it adheres to the constraints that are implicitly imposed by the training data. A fine-tuned network Φ_{ut} is obtained via unlikelihood training over a course of 100 additional epochs, with the goal of reducing the number of planarity constraint violations. For comparison, a third model Φ_{b200} , also based on Φ_{b100} , instead receives 100 epochs of regular training. All three models are evaluated by generating 300 samples each and computing certain metrics based on the number of constraint violations. Furthermore, the projected diffusion sampling procedure, where intermediate outputs are projected to their closest feasible neighbor at each diffusion step, is evaluated in an identical manner.

3.3 Constraint Violation Metrics

Each graph should satisfy the following constraints: connectivity, planarity and legal edge lengths. For each of these constraints, a metric is defined in order to measure how well a graph adheres to them. Connectivity is assessed via the number of disconnected components. For planarity, the edge crossing ratio is defined as half the number of legal edges that cross other legal edges to avoid double counting, normalized by the total edge count. Legal edge lengths are evaluated based on the number of illegal edges (i.e., an edge is considered illegal if its length $l \notin \{1, \sqrt{2}, \sqrt{5}\}$), which is again normalized by the total number of edges in the graph, resulting in the illegal edge ratio.

3.4 Projected Diffusion

Christopher et al. (2024) define projected diffusion as transforming the intermediate output that potentially violates constraints or exhibits other undesired behavior to the closest feasible point. Since numerous definitions of closeness across various domains exist, this makes the choice of a suitable distance measure highly dependent on the given problem setting. In the graph setting of this work, distance is measured in terms of the number of edge additions and removals. The projection step first filters out illegal edges by computing their lengths based on their node coordinates. This is optimal since edges with illegal lengths are strictly forbidden. Afterwards, all edge crossings are detected using a precomputed lookup table, which allows for an efficient retrieval of potentially conflicting edges. While iterating over these edge crossings, edges are removed if any of their conflict edges exist and had not been filtered out yet. Although this greedy filtering might remove too many edges, it is close to optimal and more than sufficient, considering that finding an optimal solution would be NP-hard for general graphs (Kratochvíl, 1990). Connectivity is enforced by first assigning a label to each node, indicating which component it is part of. In a Breadth-First-Search manner, each node branches out in an attempt to find new neighbors belonging to other components. Whenever a new component is reached, the required

Table 1: Constraint violation metrics averaged over 300 generated samples, comparing the baseline model at 100 epochs (Φ_{b100}), 200 epochs (Φ_{b200}), and the model fine-tuned via unlikelihood (Φ_{ut}).

Model	Disconn. Components	Edge Crossings		Illegal Edge Lengths	
	Avg. Count ↓	Avg. Count ↓	Rate (%) ↓	Avg. Count ↓	Rate (%) ↓
Φ_{b100}	0.303	15.627	13.537	0.003	0.003
Φ_{b200}	0.493	8.350	8.906	0.020	0.020
Φ_{ut}	0.447	10.853	9.892	0.007	0.006

connective edges are inserted into the original graph. This process is repeated until no disconnected components are left. While this approach ensures local optimality (i.e., finding the shortest connection between two components), global optimality is not guaranteed. Determining the minimum number of edges to connect all components is a variant of the NP-complete Steiner Tree Problem (Lu et al., 2003). Nonetheless, this greedy estimation suffices for this work. More importantly, the projection guarantees planarity, connectivity and legal edge lengths, therefore satisfying all the imposed constraints.

3.5 Unlikelihood training

The core idea of unlikelihood training is to use adverse examples and penalize the denoising model for producing similar output. In this work, the goal is to improve the pretrained model’s ability to produce planar graphs, since the outputs typically satisfied the connectivity and edge length but not the planarity constraint (see Section 4). To ensure that the pretrained model does not forget previously learned knowledge, unlikelihood training is combined with regular training. Concretely, during each training step, unlikelihood training is performed with a fixed probability $p = 0.1$; otherwise, regular training is applied. For each graph, 8 random query edges that would cross with an existing edge from the noisy ground truth graph are selected. These edges form the negative samples for which the model is penalized upon prediction. Instead of regular cross-entropy loss between predicted edge attributes and ground truth edge attributes, weighted cross-entropy loss is employed, where weights are assigned a positive scaling constant for negative samples and zero otherwise. Thus, the model is penalized solely for predicting edges that would cause an edge crossing.

Regarding the choice of parameters, the unlikelihood training probability $p = 0.1$ has intentionally been kept low since the denoising model would otherwise output vastly different graphs. Similarly, restricting unlikelihood training to just the last k diffusion steps (e.g., $k = \frac{T}{10}$) had detrimental effects on performance, and choosing fewer than 8 negative samples resulted in insufficient learning signals.

4 Results

During training the model quickly learned to generate graphs that are structurally similar to ground truth graphs and mostly adhere to the constraints implicitly imposed by the training data. As shown in Table 1, the baseline model Φ_{b100} yielded decent results, with most of the 300 sample graphs being connected and all except for one having exclusively legal edge lengths. Satisfaction of the planarity constraint, however, seems to be substantially more difficult since each sample contains several edge crossings (see Figures 4 and 5 in the Appendix).

Continuing the training for 100 additional epochs had mixed effects on the constraint satisfaction capabilities of Φ_{b200} . While the likelihood of an edge crossing another nearly halved, there are now significantly more disconnected components and the number of illegal edges also grew to six from just one. Considering that the learning objective is the generation of similar graphs, this comes as no surprise since constraint violations are not explicitly penalized as during unlikelihood training.

Fine-tuning via unlikelihood training did indeed result in significantly fewer planarity constraint violations compared to Φ_{b100} . While this seems quite promising initially, the actual improvement of Φ_{ut} pales in comparison to that of Φ_{b200} , especially when considering that the selection of negative samples leads to a slight training overhead. For the other constraints, Φ_{ut} surprisingly outperformed Φ_{b200} with slightly fewer disconnected components and just two illegal edges across the 300 samples, despite solely penalizing edge crossings.

Projected diffusion also did not yield clear improvements as initially anticipated. Projecting the final output to the closest feasible neighbor clearly provides strictly better outputs in terms of constraint satisfaction. Applying the projection operation on every intermediate graph, on the other hand, did

not necessarily result in graphs that were more similar to those from the ground truth dataset. The projected intermediate outputs were often vastly different and using these out-of-distribution samples as input for subsequent diffusion steps may cause the denoising model to produce suboptimal output. This is particularly relevant for early diffusion steps, during which the overall graph structure rapidly changes but less so for later steps, during which typically only small refinements occur. Outputs seem to also converge significantly faster, with intermediate graphs around $t = \frac{T}{2}$ looking nearly identical to the final output at $t = T$.

5 Discussion

SparseDiff seems promising for the generation of constrained graphs. The baseline performance demonstrates that even with limited training, models quickly learn to adhere to general constraints such as legal edge lengths and highly local constraints like connectivity, since edges are more likely between nodes that already connect to other nodes. Constraints such as planarity that require regional or even global knowledge, i.e., how nearby nodes are connected, seem to be more challenging, though further training may overcome this.

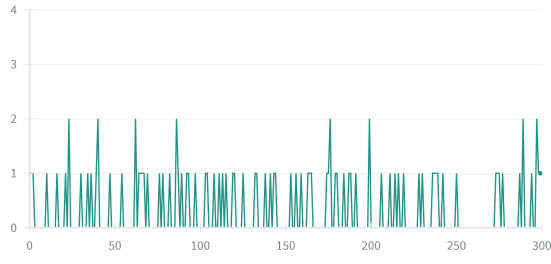
Attempts to reduce those particularly difficult constraint violations via unlikelihood training turned out less effective than just continuing with regular training. Despite this setback, future work might discover more successful approaches where constraint violations are penalized during training.

Regarding projected diffusion, projecting every intermediate output does not seem to perform strictly better than correcting only the final output graph and cannot be justified considering the additional computational overhead and the risk of suboptimal outputs due to out-of-distribution inputs. Future research should definitely evaluate the output quality using established graph similarity metrics and investigate the effects of projecting intermediate outputs during different points across the diffusion process. Viable strategies might involve applying projection at periodic intervals, during the final refinement phase or according to a time- or noise-dependent probability. Nonetheless, even for cases where projected diffusion does not lead to meaningful improvements, projection can still be utilized to ensure constraint satisfaction for final outputs.

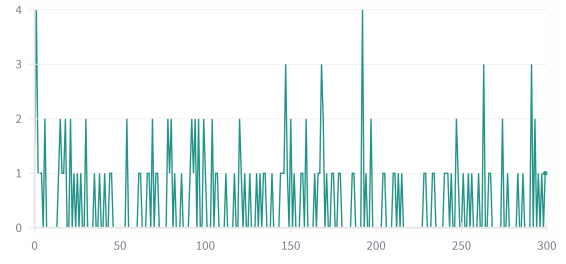
References

- Amira Alakhdar, Barnabas Poczos, and Newell Washburn. Diffusion models in de novo drug design. *Journal of chemical information and modeling*, 64, 09 2024. doi: 10.1021/acs.jcim.4c01107.
- Anonymous. But is it valid? enforcing structural constraints on graph generative models. In *Submitted to The Fourteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=LquoWQaEyw>. under review.
- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Proceedings of the 35th International Conference on Neural Information Processing Systems, NIPS '21*, Red Hook, NY, USA, 2021. Curran Associates Inc. ISBN 9781713845393.
- Jacob K. Christopher, Stephen Baek, and Ferdinando Fioretto. Constrained synthesis with projected diffusion models. *Advances in Neural Information Processing Systems 37*, 2024. URL <https://api.semanticscholar.org/CorpusID:267500184>.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- Nešetřil Jaroslav Kratochvíl, Jan. Independent set and clique problems in intersection-defined classes of graphs. *Commentationes Mathematicae Universitatis Carolinae*, 031(1):85–93, 1990. URL <http://eudml.org/doc/17810>.
- Chin Lung Lu, Chuan Yi Tang, and Richard Chia-Tung Lee. The full steiner tree problem. *Theoretical Computer Science*, 306(1):55–67, 2003. ISSN 0304-3975. doi: [https://doi.org/10.1016/S0304-3975\(03\)00209-3](https://doi.org/10.1016/S0304-3975(03)00209-3). URL <https://www.sciencedirect.com/science/article/pii/S0304397503002093>.
- Yiming Qin, Clément Vignac, and Pascal Frossard. Sparsediff: Sparse discrete diffusion for scalable graph generation. *Trans. Mach. Learn. Res.*, 2025, 2025. URL <https://api.semanticscholar.org/CorpusID:280409523>.
- Lyle Regenwetter, Giorgio Giannone, Akash Srivastava, Dan Gutfreund, and Faez Ahmed. Constraining generative models for engineering design with negative data. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=FNbv2vweBI>.
- Clément Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. Digress: Discrete denoising diffusion for graph generation. *ArXiv*, abs/2209.14734, 2022. URL <https://api.semanticscholar.org/CorpusID:252595881>.
- Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training, 08 2019.
- Yuxuan Xue, Xianghui Xie, Riccardo Marin, and Gerard Pons-Moll. Human 3diffusion: Realistic avatar creation via explicit 3d consistent diffusion models, 06 2024.
- Deyao Zhu, Mohamed Zahran, Li Erran Li, and Mohamed Elhoseiny. Motion forecasting with unlikelihood training in continuous space. In Aleksandra Faust, David Hsu, and Gerhard Neumann, editors, *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 1003–1012. PMLR, 08–11 Nov 2022. URL <https://proceedings.mlr.press/v164/zhu22a.html>.

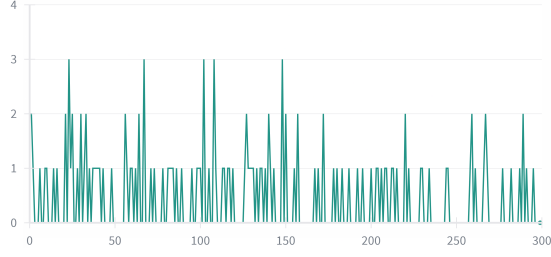
Appendix



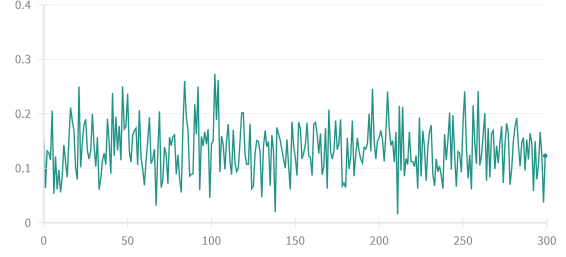
Disconnected Components: Φ_{b100}



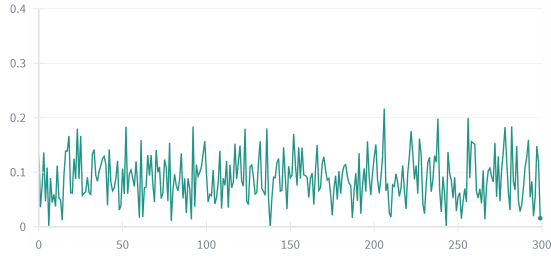
Disconnected Components: Φ_{b200}



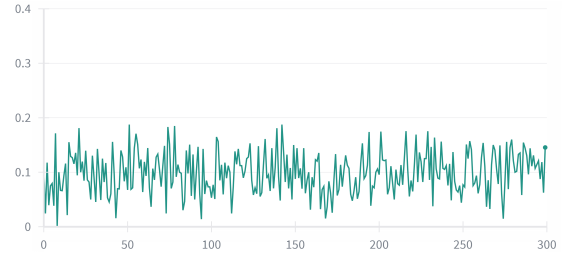
Disconnected Components: Φ_{ut}



Edge Crossing Ratio: Φ_{b100}



Edge Crossing Ratio: Φ_{b200}

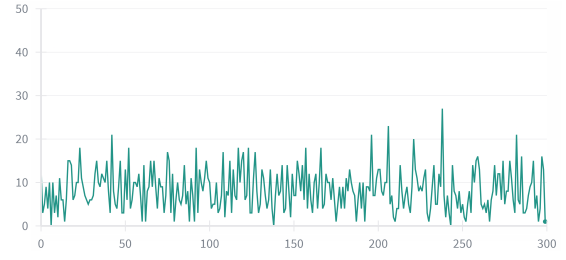


Edge Crossing Ratio: Φ_{ut}

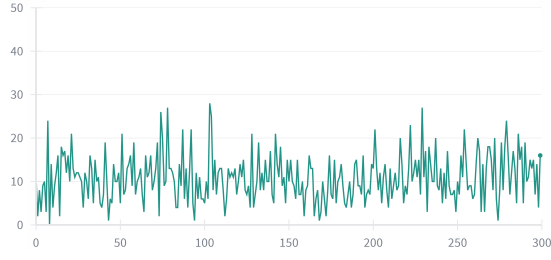
Figure 4: Disconnected Components and Edge Crossing Ratios for Φ_{b100} , Φ_{b200} , and Φ_{ut} .



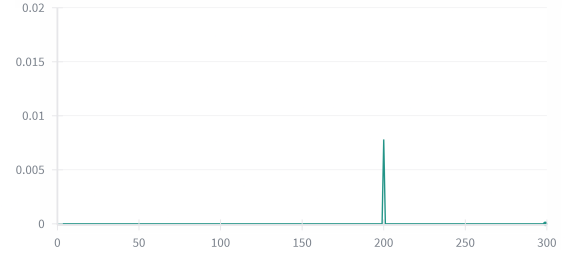
Edge Crossings: Φ_{b100}



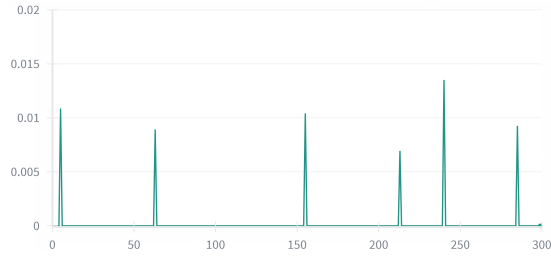
Edge Crossings: Φ_{b200}



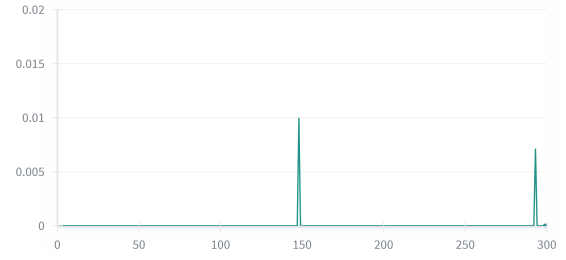
Edge Crossings: Φ_{ut}



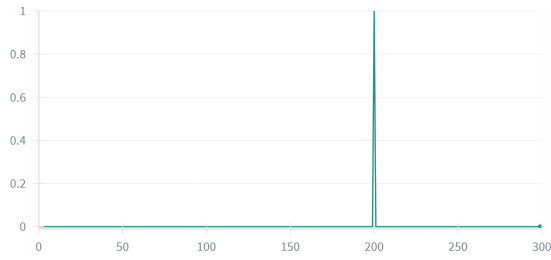
Illegal Edge Ratio: Φ_{b100}



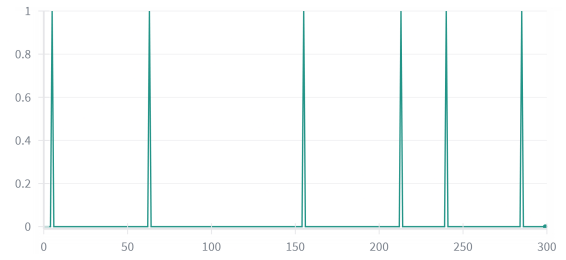
Illegal Edge Ratio: Φ_{b200}



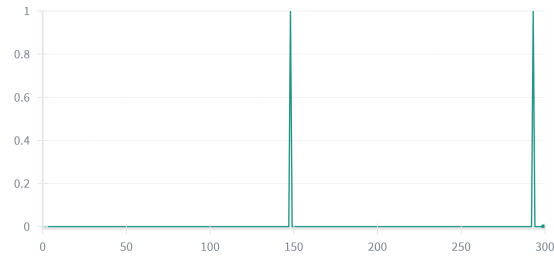
Illegal Edge Ratio: Φ_{ut}



Number of Illegal Edges: Φ_{b100}



Number of Illegal Edges: Φ_{b200}



Number of Illegal Edges: Φ_{ut}

Figure 5: Number of crossings and illegal edge counts for Φ_{b100} , Φ_{b200} , and Φ_{ut} .